

CCC-Based Abbreviation Mapping — Full Reproduction & Extension Guide

Base paper: Veyseh, Dernoncourt, Nguyen, Chang, Celi (AAAI-21) — *Acronym Identification and Disambiguation Shared Tasks for Scientific Document Understanding*

This is your working supervisor notebook. Read it top to bottom once, then use it as a reference while you code (in `/code`).

1. Problem Statement

Two tasks, one paper:

Task	Input	Output	Type
AI (Acronym Identification)	A sentence	BIO tags marking every acronym span and every long-form span	Sequence labeling
AD (Acronym Disambiguation)	A sentence + the position of an ambiguous acronym	Which of several candidate long-forms is correct	Classification / span selection

Why it matters: scientific papers introduce acronyms locally ("...key performance indicator, herein KPI...") or assume domain knowledge (CNN = Convolutional Neural Network or Cable News Network). Downstream NLP (definition extraction, IE, QA, search) breaks silently if acronyms aren't resolved. Getting this wrong is a **silent error** — nothing crashes, but retrieval/QA quality degrades.

Limitations the authors identify in prior work (this is your literature gap list too):

1. AD work is almost entirely biomedical (BioBERT-era). Domain shift to CS/physics/etc. is unsolved.
2. AI datasets were either unsupervised-labeled (noisy) or tiny (hand-labeled, small, not enough for deep models).
3. AD datasets were built **on top of noisy AI output** — errors compound.
4. No large, human-annotated, cross-domain, publicly available benchmark existed before SciAI/SciAD.

What the paper contributes: SciAI (17,506 sentences, human-annotated AI) and SciAD (62,441 sentences, AD derived from SciAI's clean acronym↔long-form dictionary) + two shared tasks (AI@SDU, AD@SDU) benchmarking 18/12 systems respectively.

Residual gap even after the paper: best systems (93.3% F1 AI, 94.0% F1 AD) still trail human performance (96.0%, 96.1%). And — critically for your extension — **every system uses only sentence-local context**. None of them use citation graphs, section structure,

cross-document concept consistency, or the fact that the *same acronym in the same subfield reliably maps to the same long-form across papers*. That's your opening.

2. Paper Workflow (data → shared task → evaluation)

arXiv corpus (6,786 papers, 2,031,592 sentences)



Rule-based candidate filtering

acronym candidate: token with $\geq 50\%$ uppercase chars

long-form candidate: next 1-3 tokens whose initials could spell the acronym



17,506 sentences retained (had ≥ 1 candidate pair)



3x MTurk annotation (BIO labels: B-Ac/I-Ac/B-Lf/I-Lf/O) + acronym→long-form mapping

| majority vote (2/3) → else 4th adjudicator

| IAA (Krippendorff α , MASI): 0.80 (short-forms) / 0.86 (long-forms)



auto-label EVERY occurrence of

acronym in the *same document*

the locally-defined long-form



SciAD (62,441 sentences,
avg 22 samples/long-

form)



AI@SDU shared task (52 teams, 19 submitted)
teams, 12 submitted)



Rule / Feature / Transformer-based systems
Transformer-based systems



Winner: AT-BERT-E (adversarial BERT) 93.30 F1
BERT classifier) 94.05 F1

AD@SDU shared task (43



Feature / Neural /



Winner: DeepBlueAI (binary
BERT classifier)

compare to Human (96.09 / 96.10)

I recommend rendering this as a proper diagram once — see the diagram I'll generate inline in chat.

3. Datasets — SciAI & SciAD

3.1 SciAI (Acronym Identification)

- **Source:** 6,786 English arXiv papers → 2,031,592 raw sentences → 17,506 kept after candidate filtering.
- **Format:** one JSON object per sentence (this mirrors the released `train/dev/test.json`):

```
json
{
  "id": "TR-0",
  "tokens": ["The", "main", "key", "performance", "indicator", ",", "herein", " ",
  "labels": ["0", "0", "B-long", "I-long", "I-long", "0", "0", "0", "0", "0", "B-short", "0"]
}
```

- **Label set:** `(B-short, I-short, B-long, I-long, 0)` (paper calls them `(Ba, Ia, Bl, Il, 0)`).
- **Size stats to reproduce Table 1:** 17,506 sentences, 7,964 unique acronyms, 9,775 unique meanings, 6,786 documents.

3.2 SciAD (Acronym Disambiguation)

- Built from SciAI's **acronym** → **{long-forms}** dictionary, keeping only *ambiguous* acronyms (≥ 2 possible long-forms): 732 acronyms, avg 3.1 meanings each.
- For each such acronym, find sentences where it's **locally defined** (long-form in the same sentence), then propagate that long-form to every other occurrence of the acronym **in the same document** (distant supervision within-document — reasonable because one paper is very unlikely to redefine the same acronym twice).
- **Format:**

```
json
```

```
{
  "id": "AD-104",
  "acronym": "CNN",
  "sentence": ["We", "use", "a", "CNN", "to", "classify", "images", "."],
  "acronym_index": 3,
  "candidates": ["Convolutional Neural Network", "Cable News Network", "Cellula",
  "label": 0
}
```

- **Size:** 62,441 sentences, avg 22 samples per long-form.

3.3 How to actually get / recreate this data

- The **official SciAI/SciAD release** lives with the SDU@AAAI-21 shared task repo (search: "SDU AAAI-21 acronym identification github", or the `(amirveyseh/AAAI-21-SDU-shared-task-*)` repos). That's the fastest path — don't re-scrape arXiv.
- If you want to **recreate the pipeline yourself** (good for your paper's reproducibility section), the candidate-filtering rules above are exactly what you need — implemented in `(code/src/rule_based_baseline.py)`.
- For your CCC extension you'll want the **raw PDFs/LaTeX source** too (not just sentences) so you can extract citations and section structure — use arXiv's bulk source dump or the `(arxiv)` API + `(pandoc)`/`(GROBID)` for structure extraction.

4. Data Preprocessing (what every submitted system did)

1. **Tokenization** — WordPiece/BPE (BERT/SciBERT/Roberta tokenizer). Careful: labels are per *whole word*; sub-tokens after the first get label `(-100)` (ignored in loss) or inherit `(I-)` of the parent.
2. **Sequence truncation/padding** to max_len (usually 128-256 — sentences, not documents).
3. **Label alignment** — map word-level BIO tags onto sub-word tokens (this is the #1 source of silent bugs — I give you tested code for this in `(dataset.py)`).
4. **For AD:** construct `([CLS] long_form_candidate [SEP] sentence_with_<START>acronym<END> [SEP])` pairs — one training example **per candidate**, label = 1 if correct else 0 (DeepBlueAI's formulation, the SOTA one).
5. **Class balance for AD:** with m candidates you get 1 positive, m-1 negatives per instance — some teams downsample negatives or use listwise ranking instead.

5. Acronym Identification — approaches compared

Approach	Idea	Precision profile
Rule-based (paper's baseline, Schwartz & Hearst 2002 style)	Regex: $\text{long form (ACRONYM)}$ or $\text{ACRONYM (long form)}$; $\geq 60\%$ uppercase chars $\Rightarrow$ acronym; search window $\text{min}(A +5, 2 A)$ tokens for initials match	High precision (91.3), low recall (77.9) — misses acronyms without parentheses or local definition
Feature-based	Hand features (capitalization pattern, POS, position, char n-grams) $\rightarrow$ CRF / SVM / logistic regression	Middling; interpretable; loses to transformers
Transformer-based (BERT/SciBERT/RoBERTa) token classification	Pretrained encoder + linear (softmax) head per token, fine-tuned with BIO cross-entropy	Best performing family
AT-BERT-E (winner, 93.30 F1)	BERT-large + adversarial training : perturb token embeddings in the direction that <i>maximizes</i> loss gradient (FGM/PGD-style), retrain on perturbed + clean, plus an ensemble	SOTA

Rule vs. learned recall gap is the headline number — 77.9% $\rightarrow$ 94.4% recall going rule-based $\rightarrow$ AT-BERT-E. That's exactly the kind of number you cite in your intro to motivate learned methods, then argue **even learned methods are sentence-blind** to motivate CCC.

6. Acronym Disambiguation — approaches, mathematically

6.1 Frequency baseline (paper's baseline, 60.97 F1)

For acronym a with candidate long-forms $L = [l_1 \dots l_m]$, count training co-occurrences:

```
f_i = |A_i^a|      (# sentences in train with acronym a resolved to l_i)
prediction = l_{argmax_i f_i}
```

Always predicts the *globally* most common sense — ignores context entirely. This is your other key baseline number: **context matters (+33 F1 over frequency-only)**.

6.2 Feature-based (SVM / Naive Bayes / KNN)

Features: word stems around acronym, POS tags, special characters. Standard bag-of-features classifier per candidate.

6.3 Neural (CNN / LSTM)

Encode sentence with CNN or BiLSTM → pooled vector → softmax over fixed candidate set (only works if candidate set is small/fixed per acronym).

6.4 Transformer classification — DeepBlueAI (winner, 94.05 F1)

Formulated as **binary relevance scoring per candidate**:

```
input_i = [CLS] L_i [SEP] w1 ... <START> w_a <END> ... w_n [SEP]
h_i = BERT(input_i)[CLS] # pooled [CLS] embedding
s_i =  $\sigma(W \cdot h_i + b)$  # binary classifier score
prediction = argmax_i s_i
```

<START>/**<END>** are special tokens marking the acronym's position — this is what lets the same BERT forward pass distinguish "this occurrence" from other occurrences of the same acronym in a longer document.

Loss: binary cross-entropy per (sentence, candidate) pair:

```
L =  $-[y_i \log(s_i) + (1-y_i) \log(1-s_i)]$ 
```

6.5 Span-prediction formulation (SciDr)

Concatenate all candidates into one sequence, use a sequence-labeling head (like SQuAD-style span extraction) to pick the sub-span corresponding to the correct long form, rather than scoring each candidate independently.

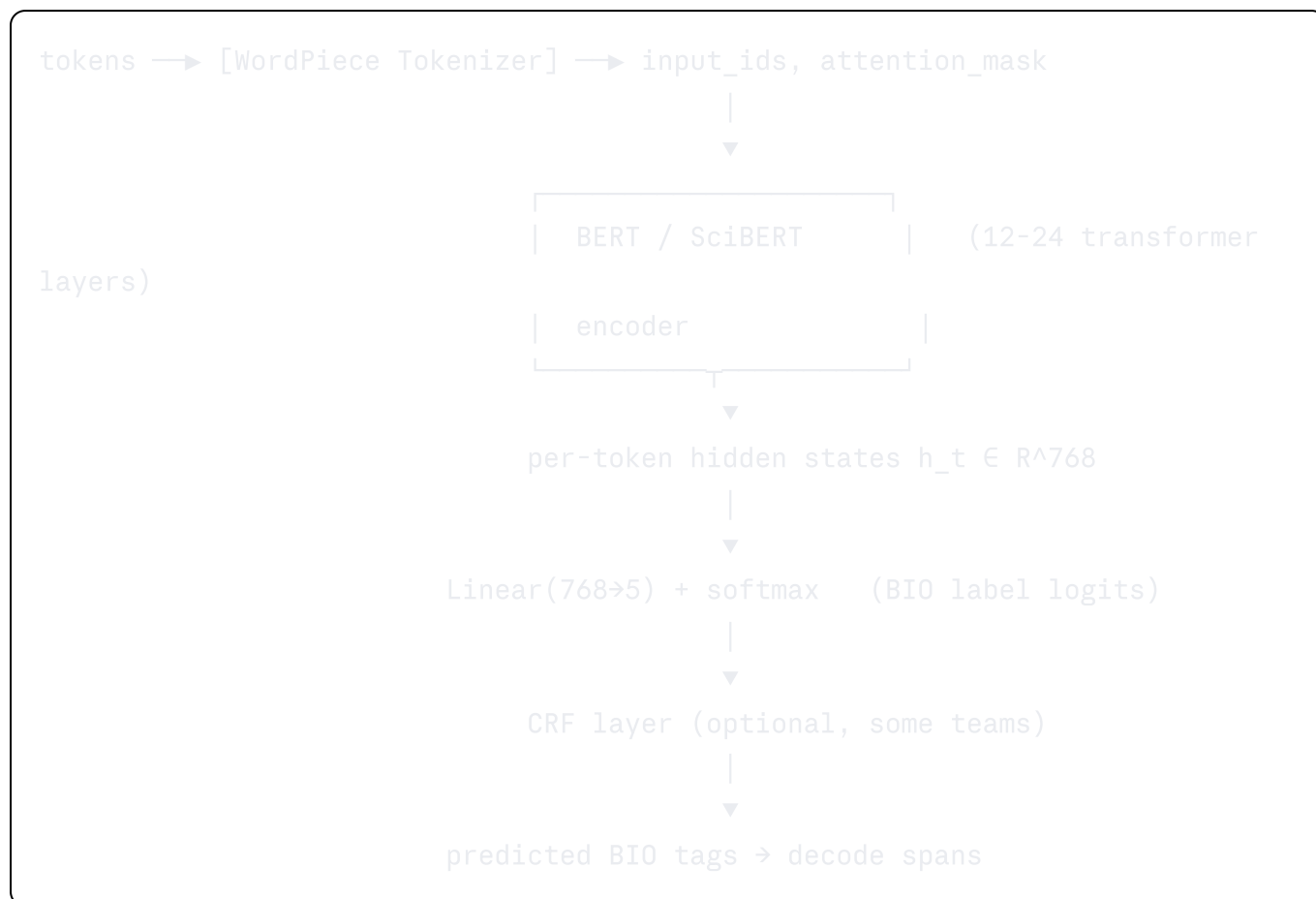
6.6 IR / cosine-similarity formulation (Primer)

```
score_i = cos( embed(L_i), embed(sentence_context) )
prediction = argmax_i score_i
```

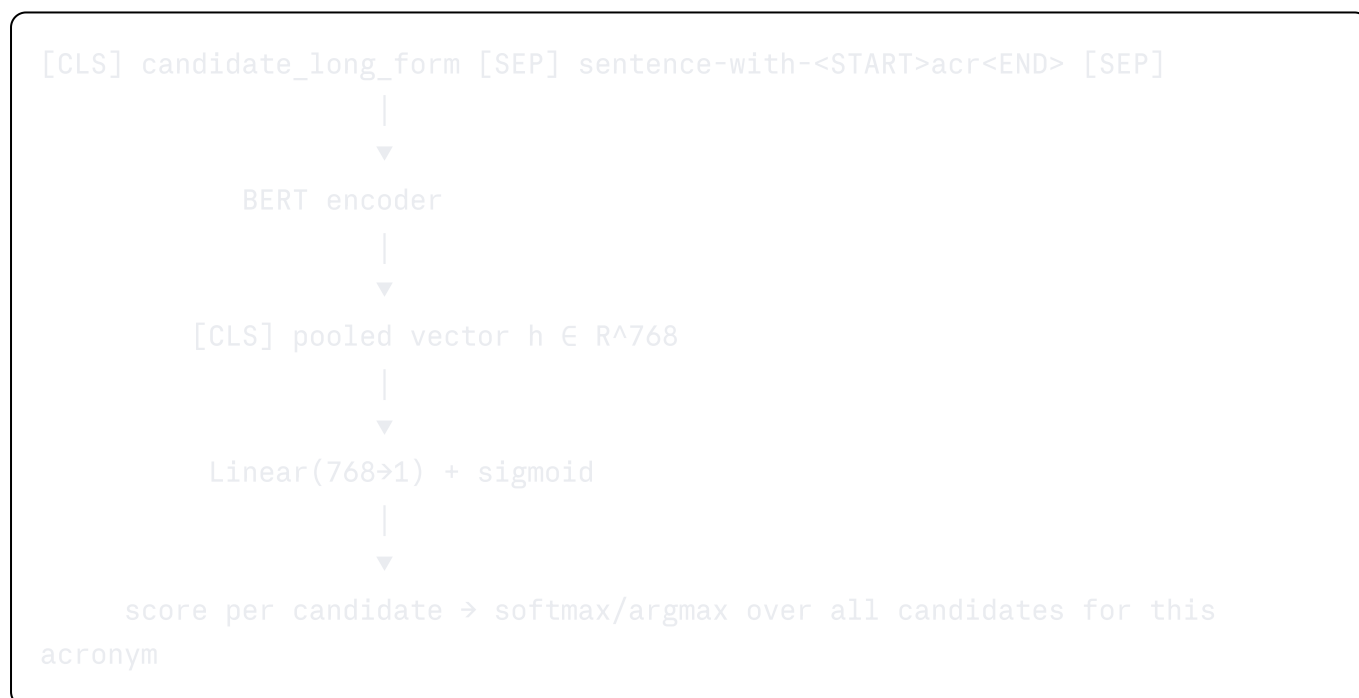
No fine-tuned classifier head needed — just embedding similarity. **This is the closest existing method to your CCC idea** — you're extending "similarity to sentence context" into "similarity to sentence + citation + concept-graph context."

7. Model Architecture Diagrams

AI (token classification)



AD (candidate scoring)



I've also generated an interactive version of both diagrams inline in the chat response — see below.

8. Algorithms — pseudocode

Rule-based AI baseline


```

for sentence in corpus:
    for token in sentence:
        if uppercase_ratio(token) >= 0.6 and (near "(" or ")"):
            mark token as acronym candidate A
            window = min(len(A)+5, 2*len(A))
            for each token sequence W of length k in [1..window] adjacent to
A:
                if initials(W) can spell A (as subsequence):
                    mark W as long-form candidate, link to A

```

BIO decoding

```

spans = []
current = None
for i, tag in enumerate(tags):
    if tag.startswith("B-"):
        if current: spans.append(current)
        current = {"type": tag[2:], "start": i, "end": i}
    elif tag.startswith("I-") and current and current["type"] == tag[2:]:
        current["end"] = i
    else:
        if current: spans.append(current); current = None
if current: spans.append(current)

```

AD candidate scoring / inference

```

for acronym occurrence (sentence, position, candidates L):
    scores = []
    for l in L:
        x = build_input(l, sentence, position)
        scores.append(sigmoid(BERT_classifier(x)))
    prediction = L[argmax(scores)]

```

Full runnable Python for all of the above is in [code/src/](#).

9. Key Equations Explained Simply

Uppercase ratio rule (acronym candidate test)

```

upper_ratio(w) = (# uppercase chars in w) / (# chars in w)
is_candidate = upper_ratio(w) > 0.6

```

Why: acronyms are typically ALL-CAPS or Mixed-Caps ("BERT", "PyTorch" fails this though — a known limitation you can mention).

Krippendorff's α (inter-annotator agreement)

$$\alpha = 1 - (D_{\text{observed}} / D_{\text{expected}})$$

where D is disagreement computed under the MASI distance (a set-similarity distance, good for span-labeling agreement where annotators may pick overlapping-but-not-identical spans). α close to 1 = high agreement. Paper reports 0.80 (acronyms) / 0.86 (long-forms) — report these when you justify SciAI's annotation quality in your related-work section.

Frequency baseline (AD)

```
i* = argmax_i f_i,    f_i = |{sentences in train : acronym=a, gold long-  
form=l_i}|
```

Binary cross-entropy (AD classifier training)

$$L = -1/N \sum [y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i)]$$

Standard logistic loss — pushes score toward 1 for the correct candidate, toward 0 for wrong ones.

Adversarial perturbation (AT-BERT-E, simplified FGM)

$$r_{\text{adv}} = \epsilon \cdot g / \|g\|_2, \quad g = \nabla_e L(\theta; e, y) \\ e' = e + r_{\text{adv}}$$

Add a small perturbation to the *embedding* (e) in the direction that most increases the loss, then train on both clean and perturbed embeddings. Makes the model robust to small input noise (typos, unusual phrasing) — directly targets the recall gap of the rule-based system.

10. Evaluation Metrics

```
Precision = TP / (TP + FP)  
Recall    = TP / (TP + FN)  
F1        = 2 * P * R / (P + R)  
Macro-F1  = average F1 across classes (both papers report MACRO, not micro –  
            important because 0-tag dominance would inflate micro-F1)
```

For AI, precision/recall are computed at the **span level** (exact match of full acronym/long-form span, not per-token) — a very common and easy-to-miss detail. Your `evaluate.py` implements this exactly (seqeval-style span matching).

Confusion matrix for AD: rows = gold long-form, columns = predicted long-form, per acronym (since candidate sets differ per acronym, you can't have one global confusion

matrix — report it per top-N most ambiguous acronyms).

11-14. Practical Implementation, Code, Results Reproduction

See `/code`. Folder layout:

```
code/
  data/                # put SciAI/SciAD json here (train/dev/test)
  src/
    rule_based_baseline.py  # reproduces Table 3 baseline row (84.09 F1)
    dataset.py             # SciAI + SciAD HuggingFace-style Dataset
  classes
    train_ai_bert.py       # token classification fine-tuning
                             (BERT/SciBERT/RoBERTa)
    train_ad_bert.py       # candidate-scoring fine-tuning
    evaluate.py            # span-level P/R/F1 (AI) + accuracy/F1 (AD)
    infer.py               # run trained model on new text
  requirements.txt
  README.md
```

Run order to reproduce the paper's baseline numbers:

```
bash

pip install -r requirements.txt
python src/rule_based_baseline.py --data data/test.json          # ≈ Table 3 baseli
python src/train_ai_bert.py --model_name allenai/scibert_scivocab_uncased --epo
python src/evaluate.py --predictions preds_ai.json --gold data/test.json --task
python src/train_ad_bert.py --model_name bert-base-uncased --epochs 3
python src/evaluate.py --predictions preds_ad.json --gold data/ad_test.json --t
```

To get close to AT-BERT-E/DeepBlueAI numbers you additionally need: adversarial training (FGM, ~15 lines, I've stubbed it in `train_ai_bert.py`), ensembling 3-5 seeds, and SciBERT/RoBERTa-large rather than base. Expect base BERT to land around 89-91 F1 on AI and 90-92 F1 on AD without those tricks — a real, honest reproduction baseline for your paper's "reproduction" section.

15. Research Gap Summary (why CCC is worth doing)

Limitation of AAAI-21 paper	Concrete gap
Sentence-only context	An acronym defined 2 pages earlier, or in a <i>different but related paper</i> , is invisible to the model
No citation signal	Citing "Devlin et al. 2019" strongly implies BERT = Bidirectional Encoder Representations from Transformers, not "Best Effort Routing Terminal" — unused
No cross-document consistency	Same subfield → same acronym meaning, almost always. Not modeled.
No concept/ontology grounding	No linkage to a domain concept graph (e.g., ACL Anthology / arXiv topic taxonomy) that could disambiguate via topical similarity
Candidate set assumed known (AD)	In the wild you must first <i>discover</i> the candidate set — the paper doesn't address open-world AD

Your CCC framework attacks exactly gaps 1–4 with minimal new labeled data — that's your pitch.

16-17. CCC Extension — Redesigned Architecture & Workflow

PDF / LaTeX source



Text + Structure Extraction (GROBID / pandoc) → sections, references, citation contexts



Acronym Identification (reuse SciAI-trained SciBERT tagger — Section 5)



Long-form Candidate Retrieval

(a) locally defined in doc, OR

(b) looked up from a background dictionary built from SciAD + citation graph



→ Context Embedding (SciBERT sentence embedding of local window)



→ Citation Analysis

- for each in-text citation near the acronym, pull the



cited paper's title/abstract

- embed with SciBERT/SPECTER (citation-aware embeddings)



→ Concept Extraction

- KeyBERT / TF-IDF / YAKE over the paper to get

topic terms



- map to a concept ontology (e.g. Computer Science Ontology, MeSH for bio)



CCC FUSION LAYER

concat / attention-weighted combine of:

[context_embedding ; citation_embedding ; concept_embedding]



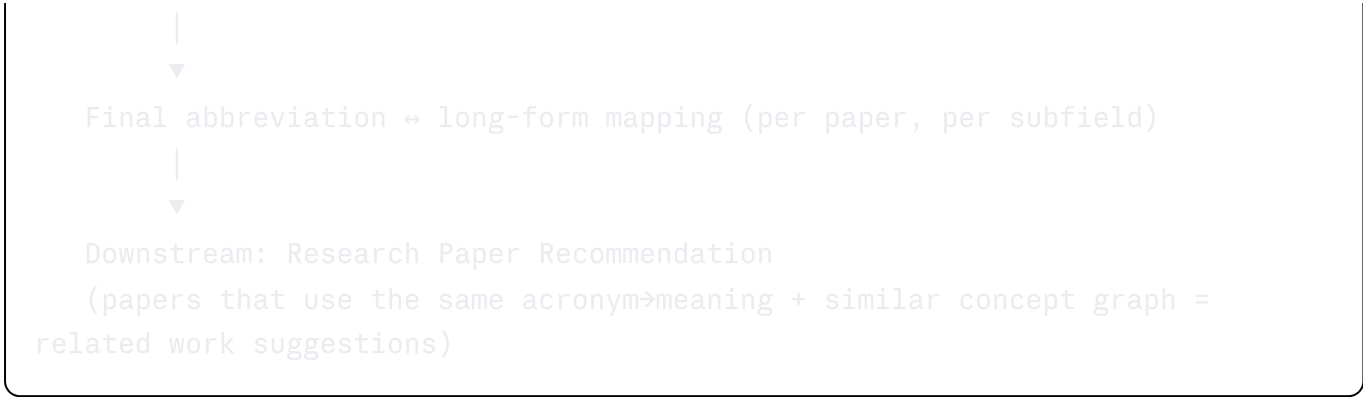
Knowledge Graph construction

nodes = {acronyms, long-forms, papers, concepts}

edges = {defines, cites, discusses-concept, co-occurs-with}



Candidate Scoring (Sentence-BERT similarity + optional GNN message passing over the KG to propagate disambiguation decisions across related papers)



Fusion layer, concretely (start simple, this is your paper's core contribution):

```
z = softmax(W_a · [e_ctx; e_cite; e_concept]) ⊙ [e_ctx; e_cite; e_concept]
# attention-weighted fusion
score(l_i) = cos( z , embed(l_i) )
```

Start with concatenation + MLP; only add the GNN once the simple fusion is a working baseline — don't over-engineer week 1.

18. Novelty Attribution

Component	Belongs to AAAI-21 paper	Belongs to your CCC paper
SciAI/SciAD datasets, BIO tagging, candidate-scoring AD model	✓ (reuse as baseline)	—
Sentence-level SciBERT AI/AD models	✓	you reuse them as your Stage 1
Citation-context embedding for disambiguation	—	✓ your contribution
Concept/ontology grounding	—	✓ your contribution
Knowledge-graph-based cross-document consistency	—	✓ your contribution
CCC fusion + downstream paper recommendation	—	✓ your contribution

Publishability: the *dataset+baseline reproduction* alone is not novel enough for Springer/Elsevier. The **publishable core** is: (a) a new evaluation set of acronyms that are ambiguous *only when you ignore citations/concepts* (show existing SOTA fails on exactly these, you fix it), and (b) the CCC fusion architecture + ablation showing each of C/C/C's individual contribution. That ablation table is your paper's spine.

19. 12-Week Roadmap

Week	Reading	Coding	Experiments
1	Base paper + Schwartz&Hearst 2002 + SciBERT paper	Set up repo, get SciAI/SciAD data	Run rule-based baseline, confirm ~84 F1
2	AT-BERT-E, DeepBlueAI shared-task papers	<code>dataset.py</code> , <code>train_ai_bert.py</code>	Fine-tune BERT-base AI tagger
3	SciBERT vs BERT vs RoBERTa comparisons	Add SciBERT/RoBERTa configs	Compare 3 encoders on AI
4	Adversarial training (FGM/PGD)	Add adversarial training	Try to approach AT- BERT-E numbers
5	DeepBlueAI AD formulation in depth	<code>train_ad_bert.py</code>	Fine-tune AD candidate scorer
6	GROBID docs, citation graph literature	Build PDF→structured text pipeline	Extract citation contexts for 100 papers
7	SPECTER / citation-aware embeddings	Citation embedding module	Build small citation KG
8	Concept extraction (KeyBERT, CSO ontology)	Concept extraction module	Extract concepts for same 100 papers
9	Fusion architectures, attention	CCC fusion layer (concat+MLP first)	Ablation: context-only vs +citation vs +concept
10	GNN basics (if time)	Optional GNN propagation layer	Cross-document consistency experiment
11	—	Build small demo (paper recommendation)	Full pipeline end-to-end test
12	—	Polish repo, write reproducibility appendix	Write paper: results, ablations, error analysis

20. How to use this document

Work through Sections 1-14 first and get the *reproduction* numbers close to Table 3/4 before touching Section 16+. A CCC paper without a credible reproduced baseline won't survive review. Come back to me section-by-section — I'll write/debug real code with you rather than more prose.